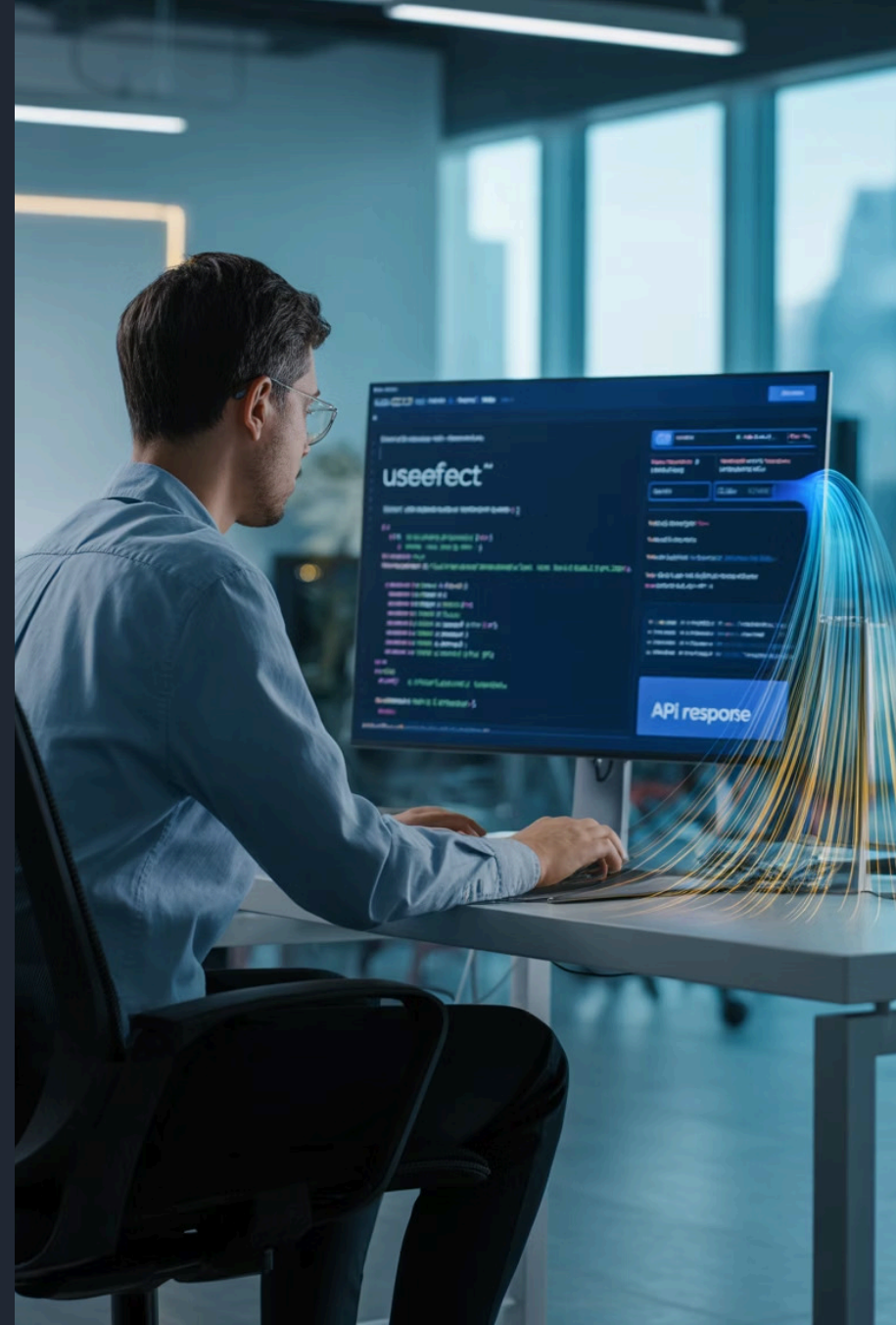


# Requisições de API no React

Um guia prático para iniciantes sobre como integrar APIs em suas aplicações React e construir interfaces dinâmicas com dados externos.



# O que é uma Requisição de API?

Uma **API (Application Programming Interface)** é uma forma de um sistema expor dados ou funcionalidades para outros sistemas consumirem. Quando fazemos uma **requisição** para uma API, estamos pedindo informações como:

- Clima de uma cidade
- Lista de filmes
- Produtos de uma loja
- Informações de usuários

No JavaScript puro, você já usou `fetch` para isso:

```
fetch("https://api.exemplo.com/dados")  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.error(err));
```



No **React**, a lógica é semelhante: buscamos dados em um servidor, mas integramos isso dentro de **componentes**, guardando a resposta no **estado** (`useState`) para atualização reativa da interface.



# Conceitos Básicos de APIs

1

## Endpoint

É o endereço específico da API que você precisa acessar para obter os dados desejados.

Exemplo: `https://api.openweathermap.org/data/2.5/weather`

2

## Métodos HTTP

Determinam o tipo de operação que você deseja realizar:

- **GET**: buscar dados (mais comum)
- **POST**: enviar dados
- **PUT/PATCH**: atualizar dados
- **DELETE**: remover dados

3

## Respostas JSON

A maioria das APIs retorna os dados em formato JSON (JavaScript Object Notation), que é facilmente manipulado no JavaScript.

4

## Chaves de API

Algumas APIs exigem autenticação com uma chave para controlar o acesso e o uso (normalmente enviada como parâmetro na URL ou no cabeçalho).



# O que Muda no React?

Em **React**, a implementação de requisições API tem algumas particularidades importantes:

## → Estado para Armazenar Dados

Usamos `useState` para guardar a resposta da API e permitir que o componente se atualize automaticamente.

## → Funções Assíncronas

Normalmente implementamos com `async/await` para lidar com a resposta de forma mais legível.

## → Renderização Automática

O React re-renderiza o componente quando o estado muda, atualizando a interface sem código adicional.



**i Lembre-se:** O conceito de requisição é o mesmo do JavaScript puro. O React apenas organiza isso dentro de componentes e usa estado para atualizar a interface automaticamente.

# Fetch vs Axios: Duas Abordagens

## Fetch (Nativo)

```
useEffect(() => {  
  const buscarDados = async () => {  
    try {  
      const response = await fetch('https://api.exemplo.com/dados');  
      if (!response.ok) {  
        throw new Error('Erro na requisição');  
      }  
      const dados = await response.json();  
      setDados(dados);  
    } catch (erro) {  
      setErro(erro.message);  
    } finally {  
      setCarregando(false);  
    }  
  };  
  
  buscarDados();  
}, []);
```

Vantagem: Nativo do navegador, sem dependências externas

Desvantagem: Precisa converter com `.json()` e verificar `response.ok` manualmente

## Axios (Biblioteca)

```
useEffect(() => {  
  const buscarDados = async () => {  
    try {  
      const response = await  
        axios.get('https://api.exemplo.com/dados');  
      setDados(response.data);  
    } catch (erro) {  
      setErro(erro.message);  
    } finally {  
      setCarregando(false);  
    }  
  };  
  
  buscarDados();  
}, []);
```

Vantagem: Já converte para JSON automaticamente (`response.data`) e trata erros de rede

Desvantagem: Precisa instalar dependência externa